Agentforce Observability

The Salesforce objects that hold agent logs, traces, errors, and audit data: what each one contains, and how to use it to troubleshoot and audit agent behavior.

What's inside

1 How the data fits together
2 Before you start: enable, access, query
3 Session Tracing objects (what the agent did)
4 Trust Layer Audit & Feedback objects (what the LLM saw)
5 Agent quality, intent, and usage objects
6 Core platform objects (config, change audit, debug logs)
7 Local preview trace files (developer loop)
8 Troubleshooting recipes
9 Gotchas and join-key cheat sheet

VERIFIED queried in a live org (`afs4019oct` and `afs40112oct`, Oct 2026) DOCS from Salesforce documentation or Trailhead; not exercised

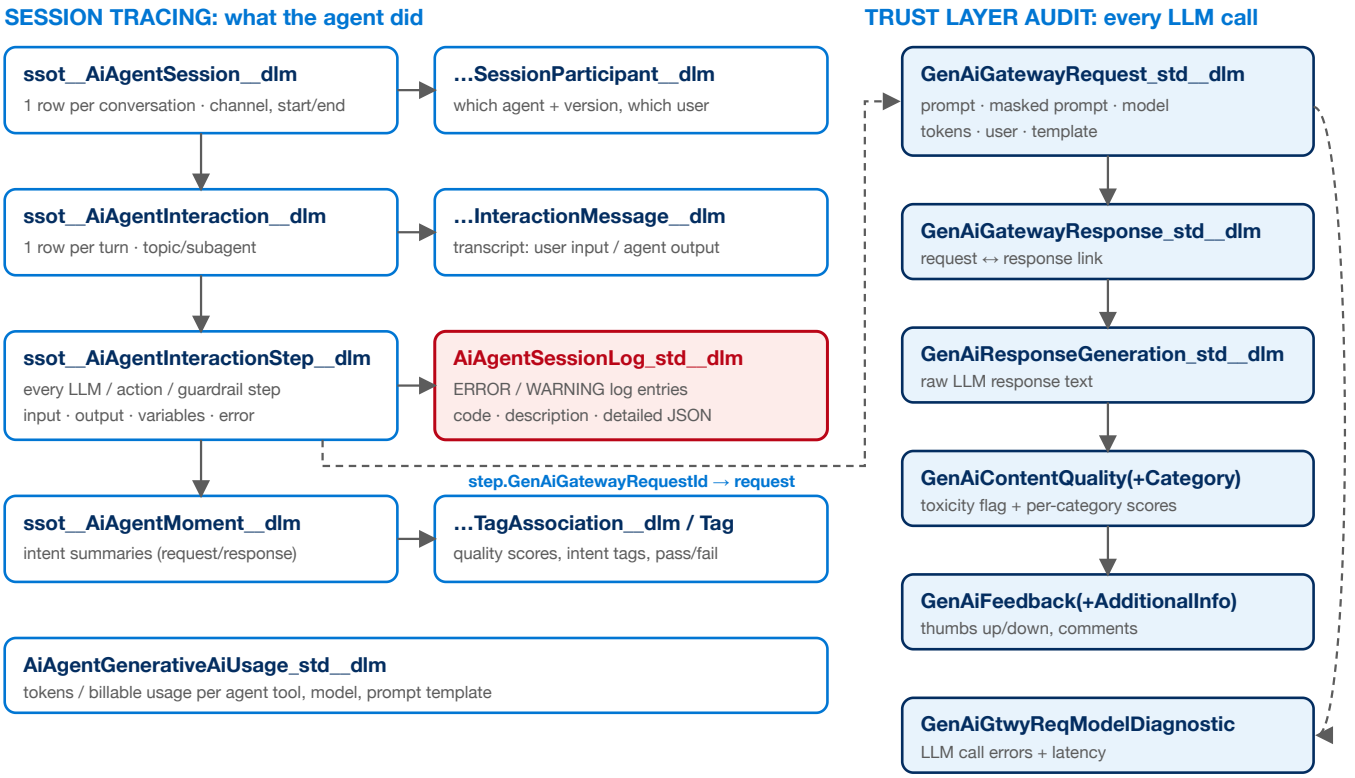
Audience	Salesforce admins, Agentforce developers, support and QA teams, auditors
Applies to	Agentforce agents built in Agentforce Builder or with Agent Script (Agentforce DX)
Requires	Data 360 (Data Cloud). Agentforce Session Tracing and Audit & Feedback turned on
Date	October 2026

1. How the data fits together

Agent activity lands in three layers:

- **Session Tracing** records *what the agent did*: the conversation, each turn, and every reasoning, action, and guardrail step.
- **Trust Layer Audit & Feedback** records *every LLM call*: the exact prompt, the raw response, toxicity scores, token counts, and user feedback.
- **Core platform objects** record *configuration and change history*: agent versions, permission changes, and Apex debug logs.

The first two layers live in Data 360 as Data Model Objects (DMOs, API names ending in `__dlm`).



Read it top-down. A **session** holds **interactions** (turns). Each interaction holds **steps** and **messages**. A step that called an LLM points to its **gateway request** in the audit layer, where you can read the exact prompt and response. Errors are written to the **session log**, keyed by session, interaction, and step.

Which object answers which question

Question	Start here
Did anything error? What and when?	`AiAgentSessionLog_std__dlm`, then `…InteractionStep__dlm.ErrorMessageText`
What exactly did the agent do on that turn?	`ssot__AiAgentInteractionStep__dlm` (ordered by timestamp)
What did the user say, and what did the agent answer?	`ssot__AiAgentInteractionMessage__dlm`
Which agent and version handled it? For which user?	`ssot__AiAgentSessionParticipant__dlm`
Which channel (Builder, Lightning, Messaging, Voice)?	`ssot__AiAgentSession__dlm`
What exact prompt went to the LLM? Was PII masked?	`GenAiGatewayRequest_std__dlm`
Was anything toxic?	`GenAiContentQuality_std__dlm` + `…QualityCategory_std__dlm`
Did users like the answers?	`GenAiFeedback_std__dlm` + `…FeedbackAdditionalInfo_std__dlm`
How many tokens or credits did it cost?	`AiAgentGenerativeAiUsage_std__dlm`, `GenAiGatewayRequest_std__dlm`
Who changed the agent or its permissions?	`SetupAuditTrail`, `BotVersion`
Why did my Apex action fail?	Apex debug logs (`ApexLog`) with a `TraceFlag` on the agent user

2. Before you start: enable, access, query

Turn data collection on DOCS

Go to **Setup → Quick Find "Einstein Generative AI" → Einstein Audit, Analytics, and Monitoring Setup**.

Setting	Fills	Notes
Agentforce Session Tracing	`ssot__AiAgent*__dlm`, `AiAgentSessionLog_std__dlm`	Required for Builder's Trace tab, Sessions & Intents, and Agent Analytics
Audit and Feedback	`GenAi*_std__dlm` / `GenAI*__dlm`	Trust Layer audit trail: prompts, responses, toxicity, feedback
Knowledge/RAG Quality Data and Metrics	Quality metrics	Needed for the Quality tab in Agent Analytics

> **Data is collected only from the moment a setting is on.** Nothing is back-filled. All three settings need Data 360, and the data consumes Data 360 credits. In `afs40112oct`, Session Tracing had data but every audit table was empty, a sign that **Audit and Feedback was off**.

Who can see it

- A Data 360 permission set (e.g. Data Cloud Admin / Data Cloud User) to browse in **Data Cloud → Data Explorer** or run SQL in **Query Editor**.
- Agentforce Studio access to use **Observe & Optimize** (Analytics, Sessions & Intents, Alerts) and the Builder **Trace** tab.
- View Setup and Configuration to read `SetupAuditTrail`.

> **The audit data is sensitive.** The prompt and response text holds conversation content and record data. Note that "data masking for LLMs is currently disabled for agents" (Trust Layer docs). Restrict Data 360 access accordingly.

Four ways to read the data

Tool	Best for	Example
Salesforce CLI (SOQL) VERIFIED	Scripting, ad-hoc investigation, CI checks	`sf data query --json -q "SELECT … FROM AiAgentSessionLog_std__dlm"`
Data Explorer / Query Editor	Browsing records, SQL joins	Data Cloud app → Data Explorer → pick the DMO
Agentforce Builder → Trace tab	One session, step by step, with timings	Open the session in Builder or Sessions & Intents
OTel API (Beta) DOCS	Export to Datadog, Splunk, or Grafana	`GET /services/data/v66.0/einstein/audit/otel/{sessionId}` (last 72 h, one session per call)

SOQL rules for these DMOs VERIFIED

- Fields use the `ssot__…__c` pattern on Session Tracing DMOs (e.g. `ssot__StartTimestamp__c`) and plain `…__c` on `_std__dlm` DMOs (e.g. `LogDateTime__c`).
- `WHERE`, `ORDER BY`, `GROUP BY` + `COUNT/SUM/MIN/MAX`, `LAST_N_DAYS:n`, `LIKE '%text%'`, and `NOT … LIKE` all work, including on the long JSON text fields.
- Empty values are stored as the string `'NOT_SET'`, not `null`. Filter with `!= 'NOT_SET'`.
- Reserved words can't be aliases. `MIN(x) first` fails; use `firstTs`.

```
sf data query --json -q "SELECT COUNT() FROM ssot__AiAgentSession__dlm WHERE ssot__StartTimestamp__c = LAST_N_DAYS:7"
```

3. Session Tracing objects: what the agent did

These DMOs give a complete, step-level record of every agent conversation in every channel: Builder preview, Lightning (employee agents), Messaging, Voice, and the Agent API. They are the production equivalent of the developer trace files.

`ssot__AiAgentSession__dlm`

Agent Session · one row per conversation VERIFIED

Field	What you learn
`ssot__Id__c`	Session ID. Matches the session ID in Builder, preview traces, and Sessions & Intents
`ssot__StartTimestamp__c` / `EndTimestamp__c`	When the conversation started and ended (duration)
`ssot__AiAgentChannelType__c`	Where it happened. Values seen: `Builder`, `LightningDesktopCopilot`, `LEX/LDS`, `MIDAS`, `NOT_SET`
`ssot__AiAgentSessionEndType__c`	How it ended (docs: resolved, escalated, deflected, …). Often `NOT_SET` for previews
`ssot__RelatedMessagingSessionId__c` / `RelatedVoiceCallId__c`	Link to the `MessagingSession` / `VoiceCall` record for service-channel conversations
`ssot__VariableText__c`	Session variables (JSON)
`ssot__PreviousSessionId__c`	The previous session, for multi-agent hand-offs

Use it to: count conversations by channel or day, find sessions in a time window, and link to the Messaging record.

`ssot__AiAgentSessionParticipant__dlm`

Who took part · agent and user rows VERIFIED

Field	What you learn
`ssot__AiAgentSessionId__c`	The session
`ssot__AiAgentSessionParticipantRole__c`	`AGENT` or `USER`
`ssot__AiAgentApiName__c`	**Which agent** (e.g. `Pronto_Employee_Agent`). `Authoring agent` = Builder's own AI assistant
`ssot__AiAgentVersionApiName__c`	**Which version** (`v0` = unpublished preview, `v1`, `v2`, …). Use it to compare versions
`ssot__AiAgentType__c`	Employee, service, and so on
`ssot__ParticipantId__c` / `ParticipantObject__c`	The user's Salesforce ID (USER rows). `PlannerPreview-…` on AGENT rows for previews

Use it to: answer "which agent and version said this, to whom?", track sessions per user, and check whether errors began with a new version.

`ssot__AiAgentInteraction__dlm`

One row per turn (user input → agent response) VERIFIED

Field	What you learn
`ssot__Id__c`	Interaction ID, the key that steps and messages hang off
`ssot__AiAgentSessionId__c`	Parent session
`ssot__AiAgentInteractionType__c`	Interaction kind (e.g. a turn)
`ssot__TopicApiName__c`	Topic or subagent that handled the turn (often `NOT_SET`; check the steps)
`ssot__StartTimestamp__c` / `EndTimestamp__c`	Turn latency
`ssot__TelemetryTraceId__c`	Distributed trace ID (OTel correlation)
`ssot__PrevInteractionId__c`	Previous turn, to rebuild the order

`ssot__AiAgentInteractionStep__dlm`

Every step inside a turn · **the most diagnostic object** VERIFIED

Field	What you learn
`ssot__AiAgentInteractionId__c`	Parent turn
`ssot__AiAgentInteractionStepType__c`	Step kind (table below)
`ssot__Name__c`	Action name (`IdentifyRecordByName`), subagent name (LLM steps), or evaluator name
`ssot__InputValueText__c`	JSON input. For actions: `{"actionName":…,"actionInput":{…}}`. Shows exactly what the LLM passed in
`ssot__OutputValueText__c`	JSON output. For actions: `actionResponse.__action_execution_status__` plus the returned data. Guardrail and grounding results appear here too
`ssot__ErrorMessageText__c`	Error for the step, or `NOT_SET`. Often just `{"message":"Unknown error"}`, so read the session log for details

`ssot__PreStepVariableText__c` / `PostStepVariableText__c`	Agent variables before and after the step, to catch wrong `set` logic
`ssot__GenAiGatewayRequestId__c` / `GenerationId__c`	**Join to the Trust Layer audit** for the exact prompt and response
`ssot__StartTimestamp__c` / `EndTimestamp__c`	Per-step latency

Step types observed

Type	Typical `Name`	Tells you
`CLASSIFIER_STEP`	`pre_orchestration.guardrail`	Router or safety classifier before reasoning. The output shows `af.router_classifier.selected_target`
`TOPIC_STEP`	subagent / topic name	Topic selection
`LLM_STEP`	subagent name, `agent_router`, `Atlas__GroundednessValidationPrompt`	A reasoning call, or the grounding check (its output has `category=GROUNDED/UNGROUNDED/VALIDATION_ERROR`)
`ACTION_STEP`	action API name	Apex, Flow, prompt template, or standard action: input, output, status
`VARIABLE_UPDATE_STEP`	`__state_update_action__`	Agent Script `set` / state changes
`TRUST_GUARDRAILS_STEP`	`InstructionAdherence`, task resolution	Quality evaluators. Output: `"InstructionAdherence: value=HIGH, explanation=…"`

The DMO reference documents different values (`UserInputStep`, `LLMExecutionStep`, `FunctionStep`). Check what your org actually stores with `GROUP BY ssot__AiAgentInteractionStepType__c`.

`ssot__AiAgentInteractionMessage__dlm`

The transcript · what the user and agent said VERIFIED

Field	What you learn
`ssot__AiAgentInteractionMessageType__c`	`Input` (user) or `Output` (agent)
`ssot__ContentText__c`	The message text. Search it with `LIKE` for failure phrases
`ssot__AiAgentInteractionMsgContentType__c` / `Modality__c`	text/plain, application/json, audio; text or voice
`ssot__MessageSentTimestamp__c`	When it was sent
`ssot__AiAgentSessionId__c` / `AiAgentInteractionId__c` / `AiAgentSessionParticipantId__c`	Session, turn, and speaker

Use it to: read conversations, and find what users actually experienced ("something went wrong", "could not find", rate-limit text). Several errors surface *only* here.

`AiAgentSessionLog_std__dlm`

Agent error log · **start every investigation here** VERIFIED

Field	What you learn
`LogLevelType__c`	DEBUG / INFO / WARNING / **ERROR** / FATAL
`Category__c`	Seen: `SYSTEM_ERROR`. Docs: Tool, LLM, Retrieval, Guardrail, Auth, HTTP, Delegation, Platform, Lifecycle, Performance
`Code__c`	Error code. Seen: `ACTION_EXECUTION_FAILED`, `LLM_STEP_FAILED`
`Description__c`	Short text, e.g. "Invalid response from Core Action Execution"
`DetailedDescription__c`	**Full JSON diagnostics.** Real example: `"Failed to retrieve the entity ID for the input value for Input:Contact. This may be due to missing or mismatched permissions."`
`AiAgentSessionId__c` / `AiAgentInteractionId__c` / `AiAgentInteractionStepId__c`	Exactly where it happened, to join back to the step
`CauseLogId__c` / `RootCauseLogId__c`	Error chains: what caused this error
`LogDateTime__c`, `TelemetryTrace__c`	Time and trace correlation

```
SELECT LogDateTime__c, Code__c, Description__c, DetailedDescription__c, AiAgentSessionId__c, AiAgentInteractionStepId__c
FROM AiAgentSessionLog_std__dlm
WHERE LogLevelType__c = 'ERROR' AND LogDateTime__c = LAST_N_DAYS:7
ORDER BY LogDateTime__c DESC
```

An empty log doesn't mean no errors. Rate-limit failures and "successful" actions that return nothing never reach this table. Always run the text searches in Recipe 8.2 as well.

4. Trust Layer Audit & Feedback objects: every LLM call

Every request through the Einstein LLM gateway is recorded: planner reasoning, prompt templates, guardrail evaluators, the grounding checker, and Builder's assistant. These DMOs show what the model was asked, what it answered, whether the content was safe, and what it cost.

> **Two naming sets.** Orgs can hold the standard `GenAi…_std__dlm` DMOs (current; fields like `TotalTokenCount__c`) and the legacy `GenAI…__dlm` DMOs (camelCase fields like `totalTokens__c`). In `afs4019oct` both sets were populated with the same 1,303 requests. Prefer `_std__dlm`.

`GenAiGatewayRequest_std__dlm`

The LLM request · legacy: `GenAIGatewayRequest__dlm` VERIFIED

Field	What you learn
`PromptText__c` / `MaskedPromptText__c`	**The exact prompt**, with and without PII masking: system instructions, conversation, and grounding data
`AiModelName__c` / `AiProviderName__c`	Model and provider (e.g. `OpenAI-gpt-4.1…`, `anthropic.claude-sonnet…`)
`AiApplicationFeatureName__c`	Which feature called it. Seen: `plannerservice` (agent reasoning), `PromptTemplateGenerationsInvocable`, `Guardrails and Citations`, `nextGenAuthoring` (Builder assistant), `agentforce-standard-evals`
`AiPromptTemplateName__c` / `AiPromptVersionName__c`	Prompt template and version behind a template action
`PromptTokenCount__c` / `CompletionTokenCount__c` / `TotalTokenCount__c`	Token usage, for cost analysis
`AiPlannerIdentifier__c` / `BotVersionIdentifier__c`	Which agent planner and version made the call
`CreatedById__c` / `CreatedDate__c` / `FeatureSessionIdentifier__c`	Who, when, and which session
`IsPiiMaskingEnabled__c`, `IsInput/OutputSafetyScoringEnabled__c`	Which Trust Layer protections were active
`ModelParametersText__c`	Temperature and other model parameters

`GenAiGatewayResponse_std__dlm`

VERIFIED

Links a request to its response.
`AiGatewayRequestId__c`, `ResponseParametersText__c`, `CreatedDate__c`

`GenAiResponseGeneration_std__dlm`

VERIFIED

The raw LLM output, before the agent's final formatting.
`GeneratedResponseText__c`, `MaskedGeneratedResponseText__c`, `AiGatewayResponseId__c`. Legacy: `GenAIGeneration__dlm`

`GenAiContentQuality_std__dlm` + `GenAiContentQualityCategory_std__dlm`

Toxicity and safety VERIFIED

Field	What you learn
`IsToxicityDetected__c`	Whether the prompt (input) or response (output) was flagged
`ContentTypeId__c`	INPUT (user/prompt) or OUTPUT (model response)
`SourceReferenceObject__c`, `AiApplicationFeatureName__c`	What was scored, and from which feature
Category: `CategoryTypeId__c`, `CategoryValueNumber__c`, `DetectionTypeId__c`	Per-category scores (toxicity, hate, violence, sexual, profanity, physical, …), 0–1, where 1 is most toxic

Use it to: review safety, investigate abuse, and confirm guardrails held. Real example: one INPUT was flagged in `afs4019oct`. The feature was `nextGenAuthoring` (Builder assistant), not a customer-facing agent.

`GenAiFeedback_std__dlm` + `GenAiFeedbackAdditionalInfo_std__dlm`

User feedback VERIFIED

Field	What you learn
`FeedbackTypeId__c`	Thumbs up or down
`FeedbackActionId__c`	Accept, modify, regenerate, decline
`RelatedAiResponseGenerationId__c`	The response the feedback is about
`FeedbackText__c` / `ApplicationFeedbackText__c` (AdditionalInfo)	The user's comment
`CreatedById__c`, `AiApplicationFeatureName__c`	Who gave it, and on which feature

Rows appear only after users click the feedback controls. Both test orgs had none.

GenAiGtwyReqModelDiagnostic_std__dlm VERIFIED

LLM call errors and latency.
`ErrorText__c`, `ReqStatusId__c`, `ReqLatencyMsNumber__c`, `AiModelName__c`, `EndpointName__c`, `RegionName__c`, `AiGatewayRequestId__c`.
Empty in both orgs, even with rate-limit errors present. Don't rely on it alone.

GenAiGatewayRequestTag_std__dlm · GenAiGatewayReqObjRec_std__dlm

Tags on a request (`TagText__c`, `TagValueText__c`) and the **CRM records used to ground** a response (`RecordId__c`, `RecordTypeName__c`, `RecordLabelName__c`, `RecordLinkText__c`). Use them to see which records the agent's answer was based on.

5. Agent quality, intent, and usage objects

AiAgentGenerativeAiUsage_std__dlm

Token and billing usage per agent call VERIFIED

Field	What you learn
`AgentDeveloperName__c`, `AgentTypeCode__c`	Which agent consumed it
`AiAgentToolName__c`, `PromptTemplateDeveloperName__c`	Which action, tool, or template
`GenAiGatewayModelName__c`, `ModelProviderName__c`	Model used
`PromptInputTokenCount__c` / `PromptCompletionTokenCount__c` / `PromptTotalTokenCount__c`	Tokens
`IsBillableIndicator__c`, `IsMeteredIndicator__c`, `UsageQuantity__c`, `MeteringSkipReasonCode__c`	Whether the call is billable and how much it counts
`AiAgentSessionId__c`, `AiAgentInteractionId__c`, `UserId__c`	Join back to the session, turn, and user

Use it to: find the most expensive agents, actions, and templates, and explain spikes in credit consumption.

ssot__AiAgentMoment__dlm + ssot__AiAgentMomentInteraction__dlm

Intent summaries VERIFIED

An AI-generated summary of what the user wanted and what the agent did: `ssot__RequestSummaryText__c`, `ssot__ResponseSummaryText__c`, `ssot__AiAgentApiName__c`, `ssot__AiAgentVersionApiName__c`, linked to interactions through `…MomentInteraction__dlm`. These power **Sessions & Intents**. The intent pipeline runs about 4–5 hours after a session closes DOCS.

ssot__AiAgentTagAssociation__dlm · ssot__AiAgentTag__dlm · …TagDefinition__dlm

Quality scores and intent tags VERIFIED

Scores and labels attached to a session, moment, or interaction: `ssot__IsPassed__c`, `ssot__ValueText__c`, `ssot__OutcomeType__c`, `ssot__AssociationReasonText__c` (why a score was given), and `ssot__SourceType__c`. Tag values (`ssot__Value__c`) come from tag definitions. Use them to filter for failed quality checks and their reasons.

ssot__AiAgentAction__dlm · ssot__AiAgentTopic__dlm

Lookups VERIFIED

Catalogs of the actions and topics/subagents (`ssot__DeveloperName__c`, `ssot__Label__c`) used to label analytics.

Dashboards built on these objects DOCS

Where	What it shows
Agentforce Studio → Observe & Optimize → **Analytics**	Effectiveness (deflection, escalation, abandonment, success rate = action steps without errors), Usage, Quality (score 1–5), Health (**error rate**, durations), Trust (adherence, toxicity), Voice, User Satisfaction
… → **Sessions & Intents**	Per-session outcome, intent summary, subagents, actions, quality score, transcript download, **Trace** tab
… → **Alerts**	Threshold alerts on any metric (e.g. error rate ≥ X%), by email or in-app, with drill-down to sessions
Agentforce Builder → Preview → **Trace** / Interaction Summary	The same steps for one conversation, with variables, timings, and errors

6. Core platform objects: configuration, change audit, debug logs

These are standard SOQL objects, not DMOs. They don't need Data 360.

Object	What you can find	Example
`BotDefinition` VERIFIED	Every agent: `DeveloperName`, `MasterLabel`, `Type`, `AgentType` (Employee, EinsteinServiceAgent, AgentforceEmployeeAgent)	`SELECT DeveloperName, AgentType FROM BotDefinition`
`BotVersion` VERIFIED	Each published version and whether it is **Active**. Every publish adds a permanent version	`SELECT BotDefinition.DeveloperName, VersionNumber, Status FROM BotVersion`
`GenAiPlannerDefinition`, `GenAiPluginDefinition`, `GenAiFunctionDefinition`	The agent's planner, topics/subagents, and actions as records (IDs seen in traces)	Map a planner ID in the audit back to an agent
`SetupAuditTrail` VERIFIED	**Who changed what, and when.** Agent permission sets created or assigned, agent user changes, field and object changes. 180-day history	"Permission set Agentforce Action Access: assigned to user EinsteinServiceAgent User"
`ApexLog` + `TraceFlag` (Tooling) VERIFIED	Apex debug logs for actions, if a trace flag is set on the **user the agent runs as** (the Einstein Agent User for service agents; the end user for employee agents)	`sf apex list log --json`, `sf apex get log --log-id …`
`EventLogFile` DOCS	Event type `InvocableAction` (API 64+): each action invocation with `INVOCATION_SOURCE`, `AGENT_ACTION`, `PLANNER_IDENTIFIER`, `BOT_SESSION_IDENTIFIER`, `DURATION`. Needs the Event Monitoring / Shield add-on	`SELECT EventType, LogDate FROM EventLogFile WHERE EventType = 'InvocableAction'`
`MessagingSession`	Service-channel conversation: `Status`, `EndedByType`, `AgentType`, message counts. Joins from `ssot__RelatedMessagingSessionId__c`	Escalation and hand-off analysis
`AiEvaluationDefinition` / test results DOCS	Testing Center and `sf agent test` definitions and results (expected vs actual subagent, actions, response). Criterion results are in `AiEvalTestCaseCritRslt`	`sf agent test run --json -n <Name> --result-format junit`

Metadata types are not SOQL objects. `GenAiPlannerBundle`, `AiAuthoringBundle`, and `GenAiFunction` can't be queried. Retrieve them with `sf project retrieve start --metadata AiAuthoringBundle:<Name>` and diff them in source control to audit what changed in the agent's logic.

Debug logs: set the trace flag on the agent's running user. `sf apex tail log` traces *your* CLI user, so it misses agent executions. Create the trace flag for the Einstein Agent User in Setup → Debug Logs. In development, `sf agent preview --use-live-actions --apex-debug --output-dir ./debug` captures Apex logs during a preview, for the Apex Replay Debugger.

7. Local preview trace files (developer loop) VERIFIED

When you preview an Agent Script agent with `sf agent preview` (or AFDX Preview in VS Code), each turn writes a full execution trace to your project. These traces have the **most detail**, including raw LLM gateway errors that Data 360 records only as "Unknown error".

```
.sfdx/agents/<Agent>/sessions/<sessionId>/
    metadata.json        planIds, mockMode ("Live Test" | "Mock")
    transcript.jsonl     user/agent turns (agent rows carry raw[].planId)
    turn-index.json      turn → trace file
    traces/<planId>.json  { planId, sessionId, topic, intent, plan: [ {type: …}, … ] }
```

Step `type`	Key fields	Errors appear as
`FunctionStep`	`function.name/input/output`, `executionLatency`	`function.errors[]` (status, detail, `urn:trace:…`)
`LLMStep`	`messages_sent` (full prompt), `tools_sent`, `response_messages`	`errors[]`, e.g. `LLM_GATEKEEPER` rate limit
`PlannerResponseStep`	`message`, `responseType`, `safetyScore`	`responseType = "Failure"`
`OutputEvaluationStep`	`category`, `reason`	`UNGROUNDED` (twice in a row = "unexpected error")
`GuardrailsStep`	`instructionAdherence`, `taskResolution`	`UNRESOLVED / UNCERTAIN`
`EnabledToolsStep`, `TransitionStep`, `VariableUpdateStep`	visible tools, routing, variable changes	missing tool = `available when` gating

CLI: `sf agent trace list` and `sf agent trace read -s <id> -f detail -d errors|actions|routing|grounding`. Project scanner: `scripts/agent-debug/scan-preview-traces.sh`.

Verified gap: `sf agent trace read -d errors` listed action failures without their message text. It returned nothing for an `LLM_GATEKEEPER` rate-limit failure. Use the scanner or `jq` on `.plan[].errors` to catch those.

8. Troubleshooting recipes

Every query below ran successfully with `sf data query --json -q "…"`. Agent errors fall into three tiers, and you need all three checks to catch everything.

Tier	What it looks like	Where it shows up	Real example found
1 · Logged	Platform recorded an error	`AiAgentSessionLog_std__dlm`, `ErrorMessageText`	Prompt-template action failed: missing permission on `Input:Contact`
2 · Hidden in text	Error text inside a step output or message; no error field set	`OutputValueText`, `ContentText`	LLM rate limit (429 / `LLM_GATEKEEPER`) during the grounding check and in Builder's assistant
3 · Silent	Status "success", but the result is wrong or empty	Action output compared with real CRM data	`IdentifyRecordByName` returned no results 9 times for contacts that exist

8.1 Tier 1: list every logged error

```
SELECT LogDateTime__c, Code__c, Description__c, DetailedDescription__c,
       AiAgentSessionId__c, AiAgentInteractionStepId__c
FROM AiAgentSessionLog_std__dlm
WHERE LogLevelType__c = 'ERROR' ORDER BY LogDateTime__c DESC

SELECT ssot__StartTimestamp__c, ssot__AiAgentInteractionStepType__c, ssot__Name__c,
       ssot__ErrorMessageText__c, ssot__InputValueText__c
FROM ssot__AiAgentInteractionStep__dlm
WHERE ssot__ErrorMessageText__c != 'NOT_SET'
```

8.2 Tier 2: find errors hidden in step outputs and messages

```
-- Gateway / rate-limit / validation errors inside step outputs
SELECT ssot__StartTimestamp__c, ssot__Name__c, ssot__OutputValueText__c, ssot__AiAgentInteractionId__c
FROM ssot__AiAgentInteractionStep__dlm
WHERE ssot__OutputValueText__c LIKE '%LLM_GATEKEEPER%'
   OR ssot__OutputValueText__c LIKE '%VALIDATION_ERROR%'
   OR ssot__OutputValueText__c LIKE '%"failed"%'

-- What users actually saw
SELECT ssot__MessageSentTimestamp__c, ssot__AiAgentSessionId__c, ssot__ContentText__c
FROM ssot__AiAgentInteractionMessage__dlm
WHERE ssot__AiAgentInteractionMessageType__c = 'Output'
  AND (ssot__ContentText__c LIKE '%went wrong%' OR ssot__ContentText__c LIKE '%unexpected error%'
       OR ssot__ContentText__c LIKE '%rate limit%' OR ssot__ContentText__c LIKE '%could not find%')
```

8.3 Tier 3: catch "successful" actions that returned nothing

```
-- Record searches that came back empty (status = success)
SELECT ssot__StartTimestamp__c, ssot__InputValueText__c, ssot__AiAgentInteractionId__c
FROM ssot__AiAgentInteractionStep__dlm
WHERE ssot__AiAgentInteractionStepType__c = 'ACTION_STEP'
  AND ssot__OutputValueText__c LIKE '%noResult%'
  AND (NOT ssot__OutputValueText__c LIKE '%searchResults%')
```

Then **check the input against real data**. In the example, the input was `{"recordName":"Alex Morgan","objectApiName":"Contact"}`, and a Contact query proved the record existed, while a SOSL search for the same names returned nothing. That points to the org's search index, not the agent. Also look for inputs the LLM invented (e.g. a case ID of `"cold-lasagna-case"`).

> **Grounding doesn't catch failed actions.** In one verified case, an action failed but the agent told the user "I'm processing compensation", and the grounding check marked that answer GROUNDED. Monitor action outputs directly.

8.4 Rebuild one turn end to end

```
-- 1. Session → turns
SELECT ssot__Id__c, ssot__StartTimestamp__c FROM ssot__AiAgentInteraction__dlm
WHERE ssot__AiAgentSessionId__c = '<sessionId>' ORDER BY ssot__StartTimestamp__c
-- 2. Turn → steps (inputs, outputs, errors, variables)
SELECT ssot__StartTimestamp__c, ssot__AiAgentInteractionStepType__c, ssot__Name__c,
       ssot__InputValueText__c, ssot__OutputValueText__c, ssot__ErrorMessageText__c, ssot__GenAiGatewayRequestId__c
FROM ssot__AiAgentInteractionStep__dlm
WHERE ssot__AiAgentInteractionId__c = '<interactionId>' ORDER BY ssot__StartTimestamp__c
-- 3. Turn → transcript
SELECT ssot__AiAgentInteractionMessageType__c, ssot__ContentText__c FROM ssot__AiAgentInteractionMessage__dlm
WHERE ssot__AiAgentInteractionId__c = '<interactionId>'
-- 4. Step → exact LLM prompt (needs Audit & Feedback)
SELECT PromptText__c, AiModelName__c, TotalTokenCount__c FROM GenAiGatewayRequest_std__dlm
WHERE Id__c = '<GenAiGatewayRequestId>'   -- legacy: GenAIGatewayRequest__dlm.gatewayRequestId__c
```

8.5 Audit agent behavior over time

```
-- Sessions per agent and version (did a new version change behavior?)
SELECT ssot__AiAgentApiName__c, ssot__AiAgentVersionApiName__c, COUNT(Id) n
FROM ssot__AiAgentSessionParticipant__dlm WHERE ssot__AiAgentSessionParticipantRole__c = 'AGENT'
GROUP BY ssot__AiAgentApiName__c, ssot__AiAgentVersionApiName__c

-- Which actions run most
SELECT ssot__Name__c, COUNT(Id) n FROM ssot__AiAgentInteractionStep__dlm
WHERE ssot__AiAgentInteractionStepType__c = 'ACTION_STEP' GROUP BY ssot__Name__c

-- Error trend by code
SELECT Code__c, COUNT(Id) n FROM AiAgentSessionLog_std__dlm WHERE LogDateTime__c = LAST_N_DAYS:30 GROUP BY Code__c

-- Toxicity hits
SELECT CreatedDate__c, AiApplicationFeatureName__c, ContentTypeId__c FROM GenAiContentQuality_std__dlm
WHERE IsToxicityDetected__c = 'true'

-- Token cost by feature and model (legacy DMO shown; _std uses TotalTokenCount__c / AiModelName__c)
SELECT feature__c, model__c, COUNT(Id) n, SUM(totalTokens__c) tokens FROM GenAIGatewayRequest__dlm
GROUP BY feature__c, model__c

-- Who changed agent permissions (Display can't be filtered in SOQL; filter the results)
SELECT CreatedDate, CreatedBy.Username, Action, Display FROM SetupAuditTrail
WHERE CreatedDate = LAST_N_DAYS:30 ORDER BY CreatedDate DESC
```

Project scripts: `scripts/agent-debug/org-agent-audit.sh [days] [org]` runs the core audit set. `scripts/agent-debug/scan-preview-traces.sh` scans local preview traces.

9. Gotchas and join-key cheat sheet

Gotchas (all observed in real orgs)

Gotcha	What to do
Empty values are `'NOT_SET'`, not null	Filter `!= 'NOT_SET'`. `!= null` matches every row
`ErrorMessageText` says only "Unknown error"	Read `AiAgentSessionLog_std__dlm.DetailedDescription__c` for the same step, or the local preview trace
Rate-limit errors never reach the error log	Search `OutputValueText` / `ContentText` for `LLM_GATEKEEPER`, `429`, `rate limit`. Trial orgs allow 100,000 LLM tokens per minute
Actions report "success" with empty or wrong results	Tier-3 checks: inspect `OutputValueText`, and compare inputs to real records
Grounding passes when an action failed	Don't treat GROUNDED as proof of success; check the action steps
Audit tables are empty while tracing works	Audit and Feedback is probably off. Turn it on (no back-fill)
Step-type values differ from the docs	Check with `GROUP BY ssot__AiAgentInteractionStepType__c`
Joining on `Id` fails ("invalid ID field")	Data 360 keys are in `Id__c` / `ssot__Id__c` (UUID strings). The SOQL `Id` is a Salesforce record ID
Two GenAI naming sets	Prefer `GenAi…_std__dlm`. Legacy `GenAI…__dlm` uses camelCase fields
`Authoring agent` and `nextGenAuthoring` rows	That's Builder's own AI assistant, not your agent. Exclude it from agent metrics
Version `v0` / channel `Builder`	Preview and test traffic. Exclude it when measuring production behavior
Data lands with a delay; intents arrive hours later	Steps and logs appear within minutes. Moments and intents take about 4–5 h; clustering runs weekly
One missing permission breaks the whole agent	The planner validates **all** actions at startup. Grant the agent user access to every action's class, flow, and template
`WITH USER_MODE` Apex returns 0 rows silently	Missing object permission. Check the debug log on the agent user, and the step output

Join keys

From	Field	To
Participant / Interaction / Message / Moment	`ssot__AiAgentSessionId__c`	`ssot__AiAgentSession__dlm.ssot__Id__c`
Step / Message	`ssot__AiAgentInteractionId__c`	`ssot__AiAgentInteraction__dlm.ssot__Id__c`
Session log	`AiAgentSessionId__c` / `AiAgentInteractionId__c` / `AiAgentInteractionStepId__c`	Session / Interaction / Step `ssot__Id__c`
Step	`ssot__GenAiGatewayRequestId__c`	`GenAiGatewayRequest_std__dlm.Id__c` (legacy: `gatewayRequestId__c`)
Gateway response	`AiGatewayRequestId__c`	Gateway request
Generation	`AiGatewayResponseId__c`	Gateway response
Quality category	`AiContentQualityId__c`	`GenAiContentQuality_std__dlm`
Feedback	`RelatedAiResponseGenerationId__c`	Generation
Session	`ssot__RelatedMessagingSessionId__c`	`MessagingSession.Id`
Participant (USER)	`ssot__ParticipantId__c`	`User.Id`
Usage	`AiAgentSessionId__c` / `AiAgentInteractionId__c`	Session / Interaction

Sources

Live verification: `sf sobject describe` and `sf data query` against orgs `afs4019oct` and `afs40112oct`, October 2026. Salesforce Developer docs: Agentforce guide (`developer.salesforce.com/docs/ai/agentforce/guide/`, including the OTel API, Agent DX preview and troubleshooting, and Agent API troubleshooting) and the Data 360 DMO reference (`developer.salesforce.com/docs/data/data-cloud-dmo-mapping/guide/`). Trailhead: Agent Analytics and Monitoring; Agentforce Testing Tools and Strategies; The Einstein Trust Layer; Create an Agent Using Pro-Code Tools; Agent Customization with Apex / Flows. Salesforce Help: Set Up Agent Analytics; Session Trace; Audit & Feedback; Monitor Agent Health. Field lists and behavior can change between releases. Re-check with `sf sobject describe -s <DMO>`.